YOUR DESK

Report

Project Members:

Sara Amri 2311279

Roya Bawazir 2310940

Layal Abdulrahman 2313114

Lama Alsherhri 2311250

Albatool Alahmari 2311041

جامعة جدة
University of Jeddah

Table of Contents

Project Definition: 5

Problem Definition: 5

Proposed Solution: 5

Scope: 6

Users and Stakeholders: 6

Tools Setup: 6

Functional & Non-Functional Requirements 7

Use case Diagram: 8

Class Diagram: 9

Sprint one 10

Project Initiation 11

Prioritized Functional and non-functional requirements: 11

Story Backlog 12

Use Case Diagram: 13

Sequence Diagram: 14

Sprint Meeting 15

Sprint Meeting 16

Sprint Test Cases 17

Sprint Test Cases 19

Sprint two 21

project Initiation 22

Prioritized Functional and Non-Functional Requirements: 22

Story Backlog 23

Use Case Diagram: 24

Sequence Diagram: 25

Sprint Meeting 26

Sprint Meeting 27

Sprint Test Cases 28
Sprint Test Cases 30
Sprint three 32
project Initiation 33
Story Backlog 34
Use Case Diagram: 35
Sequence Diagram: 35
Sprint Test Cases 36
App Screenshots 38
Code 48

Sprint zero

Project Definition:

The proposed project is an application called YourDesk, designed as a digital personal assistant for university students to help them organize their time and manage their academic life more effectively. The app acts as a centralized platform that combines all the essential study-related tools in one place, such as schedules, lectures, exams, assignments and notes. The main objective of YourDesk is to provide students with a practical and user-friendly tool that enables them to keep track of their commitments, avoid missing deadlines, and stay organized throughout the semester.

Problem Definition:

University students often face difficulties managing multiple courses, lectures, assignments, exams, and personal notes. They usually depend on separate tools (like calendars, reminder apps, and note apps), which can be confusing and inefficient. This lack of organization leads to missed deadlines, poor preparation, and reduced productivity

Proposed Solution:

YourDesk provides an all-in-one digital solution to tackle these challenges:

- Course & Schedule Management: Students can input their courses and schedule, which are displayed in a clear and structured timetable.
- Notifications: The app sends reminders for upcoming lectures, exams, and deadlines.
- To-Do List: Helps students manage tasks such as assignments and study sessions.
- Notes Section: Allows saving text notes or images for study purposes.

Scope:

Our main focus in developing YourDesk will be on the following features :

- Schedule Management: Add courses with date and time, including labs and lectures, and display them in a structured timetable.
- Notifications & Reminders: Send push notifications for upcoming lectures, deadlines, exams, and events.
- To-Do List: Create and track tasks such as assignments, projects, notes, and study sessions
- User-Friendly Interface: Provide a simple and intuitive design optimized for students' daily use.

Users and Stakeholders:

Stakeholders:

Primary Stakeholders: University students who need and use the app daily. Secondary Users: Future students or individuals with similar study needs who can also benefit. Project supervisor/Instructor: Approves and guides the project.

Users:

(Hands-on): Students (Primary & Secondary Users): The main beneficiaries of the app. University Faculty & Staff: May benefit indirectly from more organized students and reduced scheduling conflicts.

Tools Setup:

IDE: visual studio(dart/flutter)

Agile planning: JIRA

Modeling: Lucid char

Git: GitHub

Messaging: Slack

Backend: Node.js

Databased: Firebase

جامعة جدة
University of Jeddah

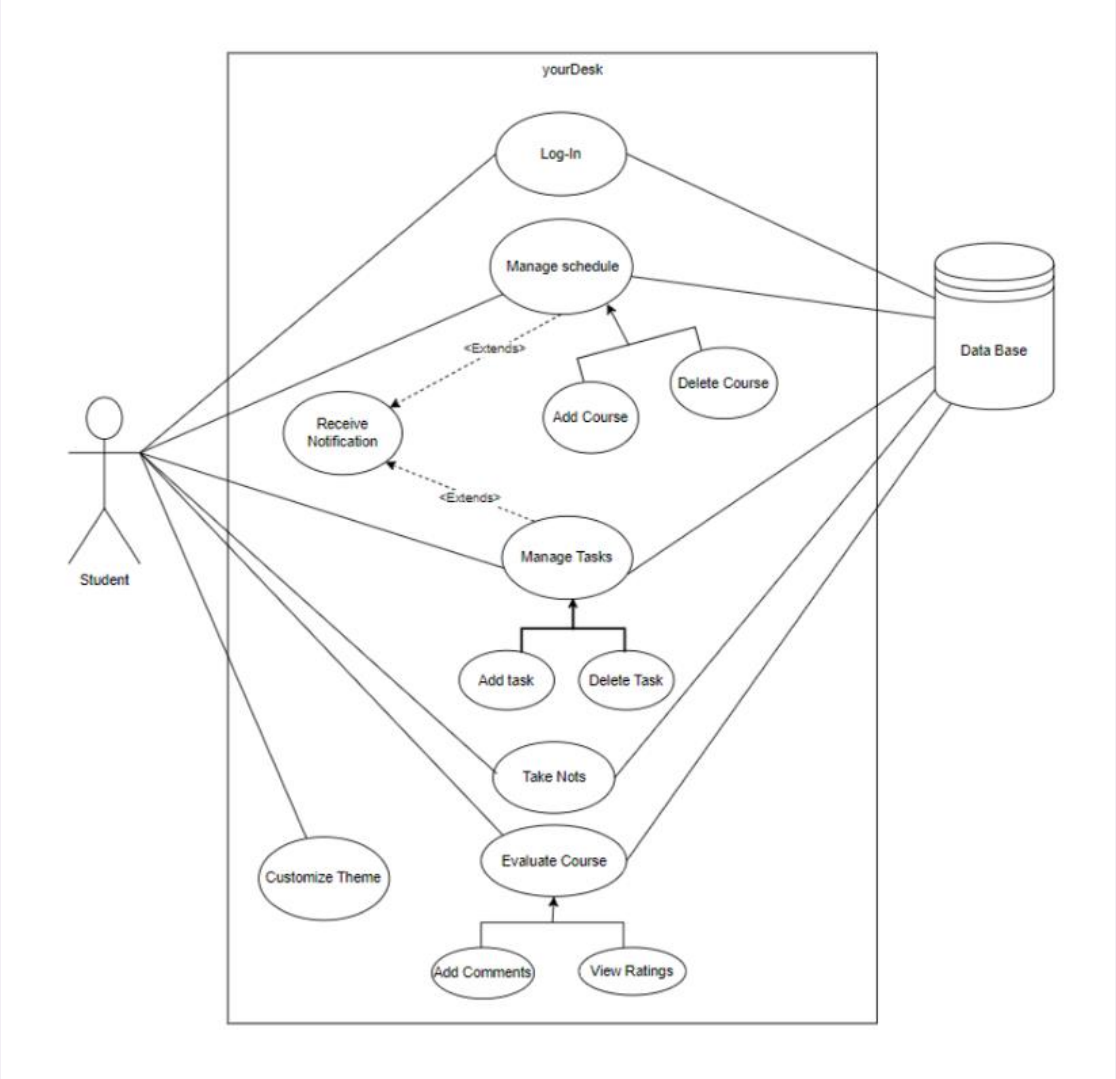
Functional & Non-Functional Requirements

Priority:

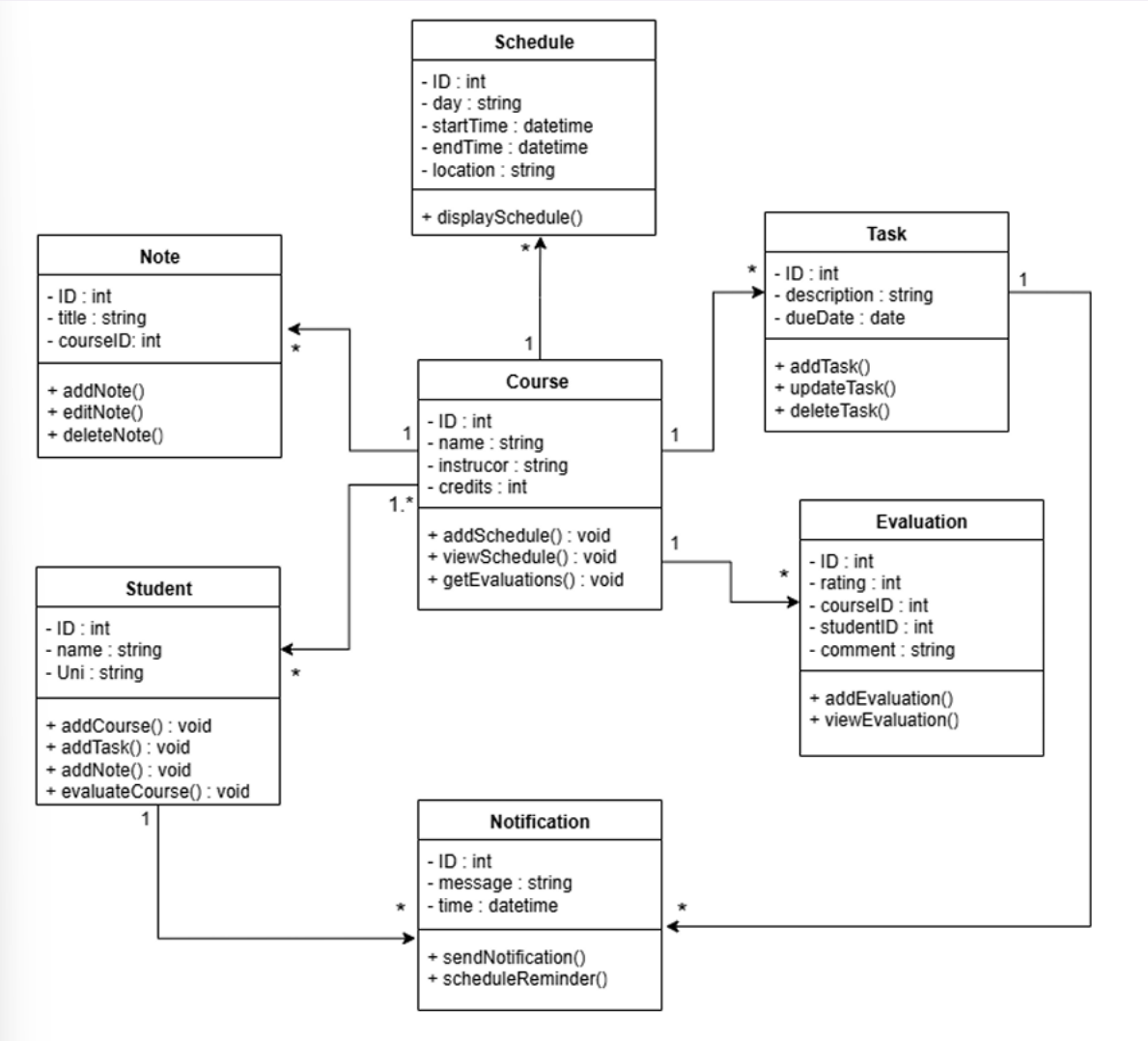
- Red (High)
- Yellow (Medium)
- Blue (Low)

ID	Requirements Definition
FR1	The system shall allow students to add, edit, and delete courses, lectures, labs, and exams with date, time, and location.
FR2	The system shall display schedules in a structured timetable view (daily, weekly, or monthly) .
FR3	The system shall send push notifications for upcoming lectures, exams, deadlines, and tasks.
FR4	The system shall allow users to create, update, and delete tasks (assignments, study sessions, projects).
FR5	The system shall automatically mark completed tasks and archive them.
FR6	The system shall allow students to create text notes and attach images/files
FR7	The system shall provide dark mode and light mode theme selection.
FR8	The system shall back up and sync the user's data with their account
NFR1	The system shall load the timetable and tasks within 2 seconds.
NFR2	The system shall deliver notifications in real-time
NFR3	The system shall provide a simple, intuitive interface optimized for daily student use.
NFR4	The system shall auto-save user data after every action to prevent data loss.
NFR5	The system shall be compatible with both iOS and Android platforms
NFR6	The system shall ensure that notifications work even if the app is closed.

Use case Diagram:



Class Diagram:



Sprint one

Project Initiation

Project Name: Your Desk

Project Members: Roya – Layal – Lama – Albatol – Sara

Project Description: In Sprint 1 of our project YourDesk, we focused on two main functions:

- Course and Schedule Management (FR1)
- Task Management (FR4).

The first allows students to add, edit, and delete courses, lectures, labs, and exams with their dates, times and location, giving them an organized timetable.

The second enables students to create, update, and delete tasks such as assignments and study sessions, helping them stay on top of their deadlines. By completing these functions, the application now provides students with a simple and practical way to manage both their schedules and tasks in one place.

Prioritized Functional and non-functional requirements:

Functional Requirements:

- FR1 (High): The system shall allow students to add, edit, and delete courses, lectures, labs, and exams with date, time, and location.
- FR4 (High): The system shall allow users to create, update, and delete tasks (assignments, study sessions, projects).

Non-Functional Requirements:

- NFR1 (High): The system shall load the timetable and tasks within 2 seconds.
- NFR3 (High): The system shall provide a simple, intuitive interface optimized for daily student use.
- NFR4 (High): The system shall auto-save user data after every action to prevent data loss.

Story Backlog

Component Name: Course Management

Story Name: Course Management

Story Sequence No: 001

Story Short Description: Students can add, edit, and delete courses with time and date.

Story Long Description: Students shall be able to input course details including course name, date, time, and location. The system shall save this data in the timetable, allow editing for corrections, and support deletion of outdated courses.

Component Name: Task Management

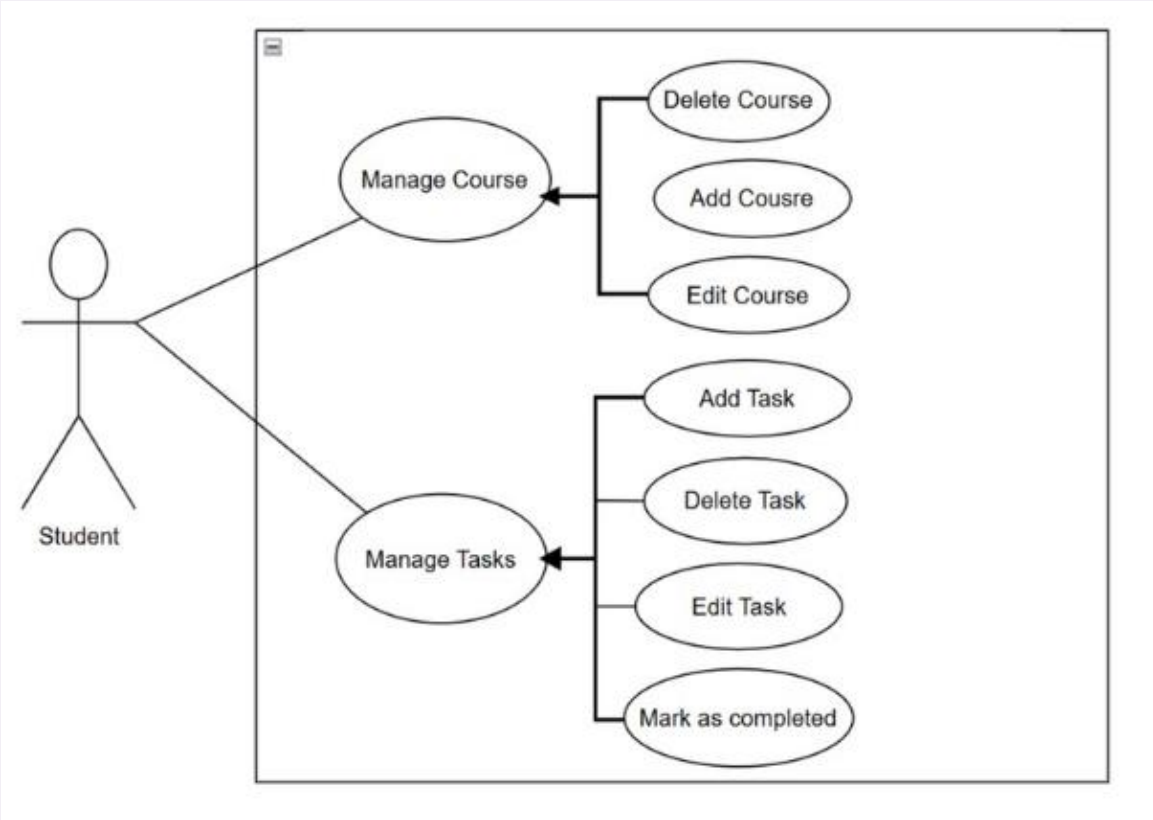
Story Name: Task Management

Story Sequence No: 002

Story Short Description: Students can manage their task list by adding, editing, deleting, and marking tasks as completed.

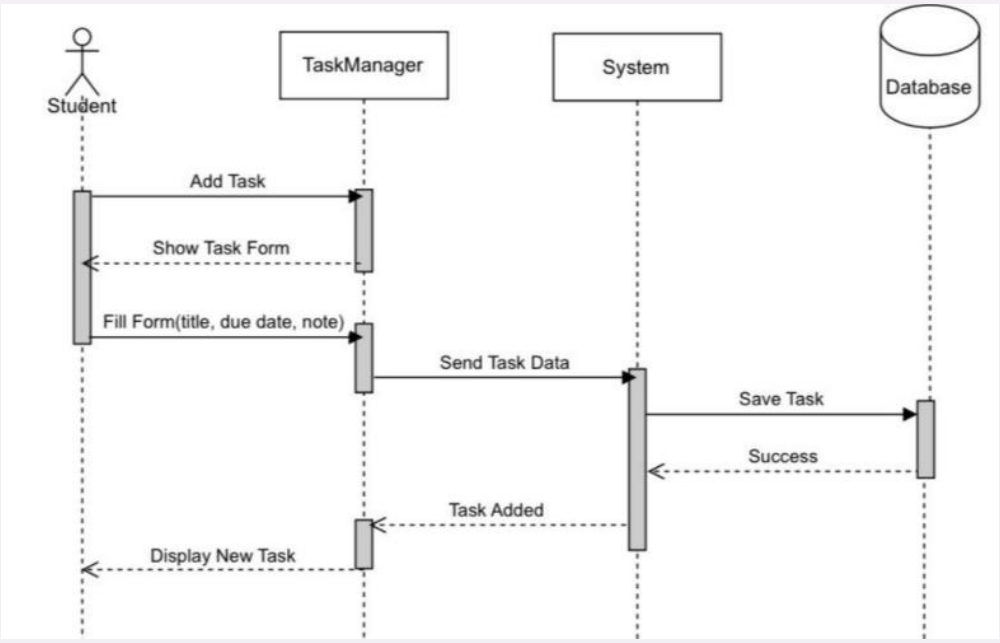
Story Long Description: Students shall be able to create tasks by entering a task name, deadline, and note. The system shall store the tasks, allow editing when details change, enable users to delete tasks once they are no longer needed, and provide a check option to mark tasks as completed for better progress tracking.

Use Case Diagram:

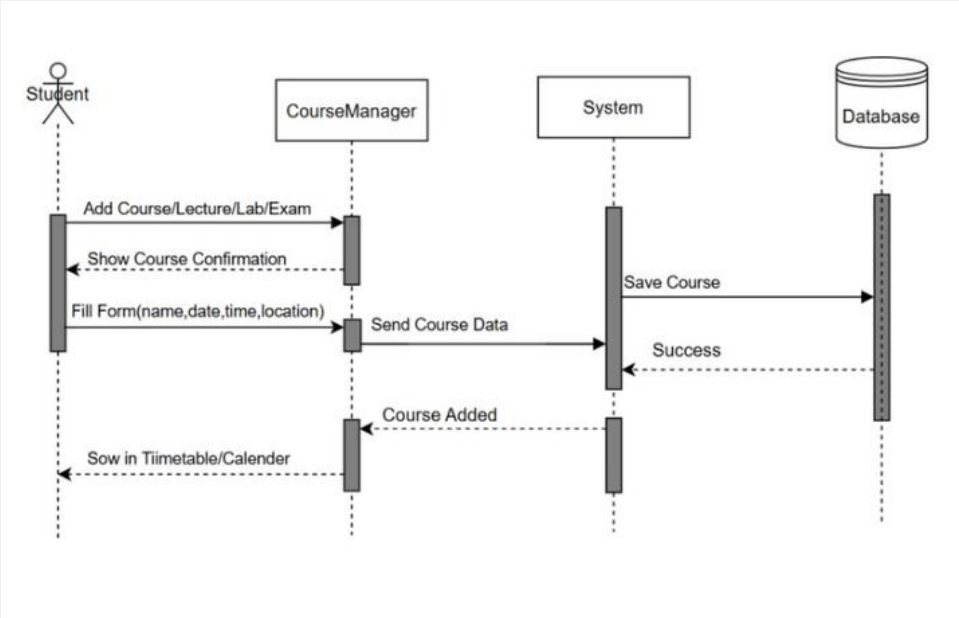


Sequence Diagram:

1-



2-



Sprint Meeting

Project Name: Your Desk

Project Members: Roya – Layal – Lama – Albatol - Sara

Sprint: i Stand up Meeting[28/9/2025]

Sprint Duration: 2 weeks

Scrum Master: Dr. Malak

Client: Roya – Layal – Lama – Albatol - Sara

Pair Programmers: Roya – Layal – Lama – Albatol - Sara

Component Name	Story Sequence Number	Use Cases (e.g., functionalities)
Course Management	001	Students can add, edit, and delete courses with time and date
Task Management	002	Students can manage their task list by adding, editing, deleting, and marking tasks as completed

Follow-up meeting questions:

1. **What has been completed since the last meeting?** After finishing Sprint 0, we made some adjustments to it and then started working on the Initiation phase of Sprint 1. After that, we began coding, starting with the basic setup
2. **What are you going tobe working on next?** We will continue developing the course management feature and begin the task management functions.
3. **Do you have any issues/impediments?** Yes, we faced some issues linking the added courses to the calendar and also had another difficulties.

Scrum's Master commentsbased on the above questions:

Sprint Meeting

Project Name: Your Desk

Project Members: Roya – Layal – Lama – Albatol - Sara

Sprint: i Stand up Meeting[5/10/2025]

Sprint Duration: 2 weeks

Scrum Master: Dr. Malak

Client: Roya – Layal – Lama – Albatol - Sara

Pair Programmers: Roya – Layal – Lama – Albatol - Sara

Component Name	Story Sequence Number	Use Cases (e.g., functionalities)
Course Management	001	Students can add, edit, and delete courses with time and date
Task Management	002	Students can manage their task list by adding, editing, deleting, and marking tasks as completed

Follow-up meeting questions:

1. **What has been completed since the last meeting?** We successfully implemented both course management (add/edit/delete courses with details) and task management (add/edit/delete tasks and mark them as completed).
2. **What are you going to be working on next?** We will focus on writing the test cases for Sprint 1.
3. **Do you have anyissues/impediments?** No, the system is working fine and the main functionalities are completed.

Scrum's Master commentsbased on the above questions:

Sprint Test Cases

Project Name: Your Desk

Project Members: Roya – Layal – Lama – Albatol – Sara

Sprint 1 Test Cases [7/10/2025]

- **Test Case Name:** [Sprint 1 – Course Management]
- **Test Case Id:** [Your Desk – Course Management]

Test Case No	Test Case Description	Expected Results	Outcome (Pass/ Fail/Other)
TC001	User enters a valid course name, location, selects at least one day, chooses start and end time, then taps Add Course	Course should be added and displayed under the selected day with the correct detalls (name, time, location)	pass
TC002	User selects multiple days for the same course arid taps Add Course	Course should appear under each of the selected days with identical details	pass
TC003	User taps Edit on an existing course, changes details (eg, location or tine), then saves	Updated details should replace the old ones in the course card	pass
TC004	User taps Delete on a course	Course should be removed from the list and calendar	pass
TC005	User enters a course name shorter than the minimum required characters	System should show a validation message and	pass

		disable the Add Course button	
TC006	User leaves the location field empty and tries to add	System should show a validation message asking for location and prevent adding	pass
TC007	User doesn't select any day and tries to add	System should show a validation message asking to select at least one day and prevent adding	pass
TC008	User sets start and end time using the time picker	Selected times should appear correctly and be saved with the course	pass
TC009	After adding a course, user opens the calendar tab	Calendar should show an indicator on the selected day and display the course details when tapped	pass
TC0010	User adds multiple courses on the same day	All courses should appear under that day's section without overwriting each other	pass

Sprint Test Cases

test Case Name: [Sprint 1 – Task Management]

Test Case Id: [Your Desk – Task Management]

TestCase No	Test Case Description	Expected Results	Outcome (Pass/ Fail/Other)
TC001	User enters a valid task name, selects due date, optionally adds a note, then taps Add Task	Task should be added under the selected date header with correct details	pass
TC002	User enters a task name shorter than the minimum required characters	System should show a validation message and disable the Add Task button	pass
TC003	User leaves the due date empty and tries to add	System should show a validation message asking to select a due date and prevent adding	pass
TC004	User marks a task as completed by checking the box	Task should be visually marked as completed	pass
TC005	User unchecks a previously completed task	Task should return to 'not completed' state	pass
TC006	User edits a task (title, due date, or note) and saves changes	Task should update in the list, and if due date changed, it should move under the correct date header	pass

TC007	Task should update in the list, and if due date changed, it should move under the correct date header	Task should be removed and no longer appear under its date header	pass
TC008	User adds multiple tasks with different due dates	Each task should appear under the appropriate date header	pass
TC009	User tries to add a task without meeting all requirements (valid name and due date)	Add Task button remains disabled	pass
TC0010	User navigates to other tabs (Calendar, Courses) and returns to Tasks	Tasks should still appear correctly under their due dates	pass

Sprint two

project Initiation

Project Description: In Sprint 2 of our project Your Desk, we focused on two main functions:

- **Schedule Manager(FR2)**
- **Notification Manager(FR3)**

The first allows students to view their academic schedules in a structured timetable format, showing their lectures, labs, and exams clearly across daily and monthly views. This helps users stay organized by seeing all their academic events arranged by date and time in one place.

The second provides students with real-time alerts for upcoming lectures, exams, deadlines, and tasks. These instant notifications ensure that students receive timely reminders, helping them stay on track and never miss important academic events.

Prioritized Functional and Non-Functional Requirements:

Functional Requirements:

• FR2 (High): The system shall display schedules in a structured timetable view (daily, weekly, or monthly).

• FR3 (High): The system shall send instant notifications for upcoming lectures, exams, deadlines, and tasks.

Non-Functional Requirement:

• NFR2 (High): The system shall deliver notifications in real-time.

Story Backlog

Component Name: Schedule Manager

Story Name: Schedule Manager

Story Sequence No: 003

Story Short Description: Students can view their schedules in a structured timetable (daily, weekly, or monthly).

Story Long Description: The system shall allow students to visualize their courses, labs, and exams in an organized timetable view. Students can switch between daily, weekly, and monthly modes to better manage their academic workload. The timetable shall automatically arrange items by date and time, ensuring a clear and accessible display for users

Component Name: Notification Manager

Story Name: Notification Manager

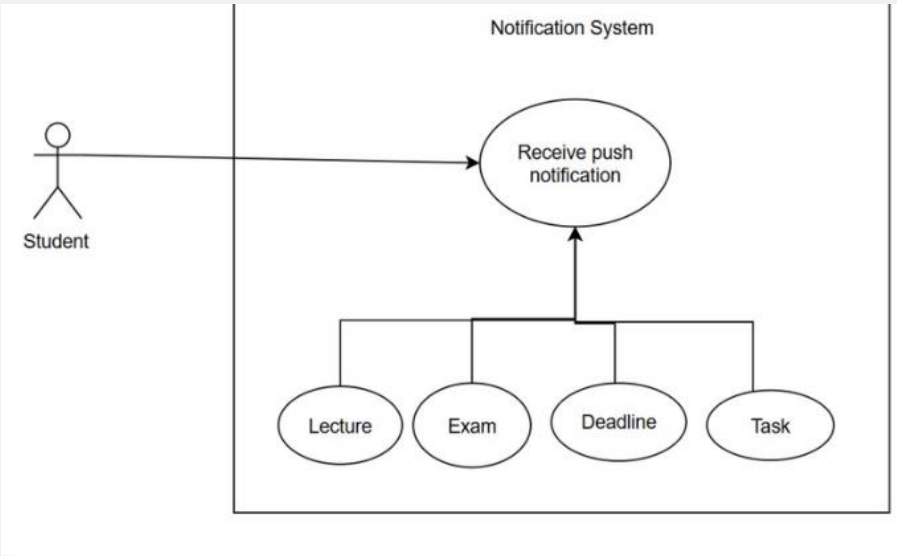
Story Sequence No: 004

Story Short Description: Students receive instant notifications for upcoming lectures, exams, deadlines, and tasks.

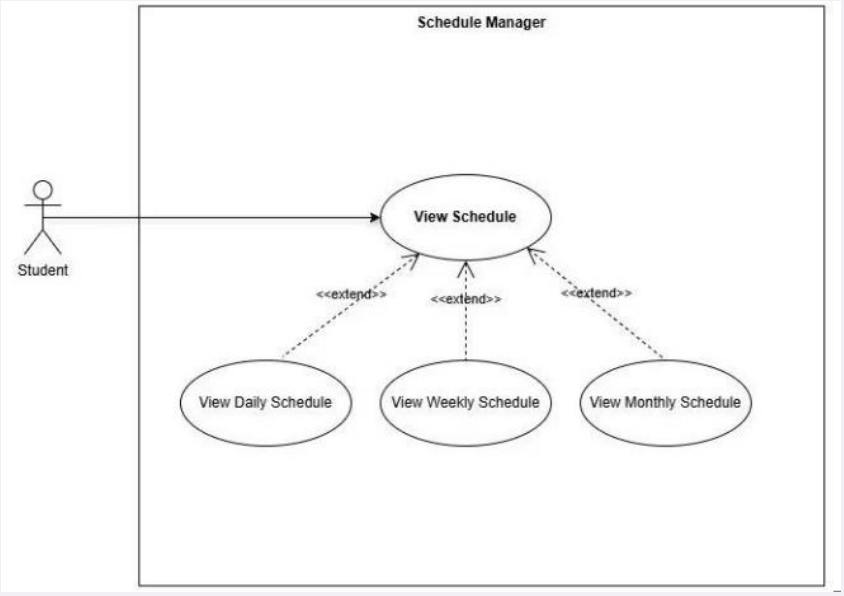
Story Long Description: The system shall send real-time notifications to remind students of upcoming academic events such as lectures, labs, exams, and assignment deadlines. Notifications shall appear instantly when an event is near, ensuring students stay aware of important schedules

Use Case Diagram:

1-

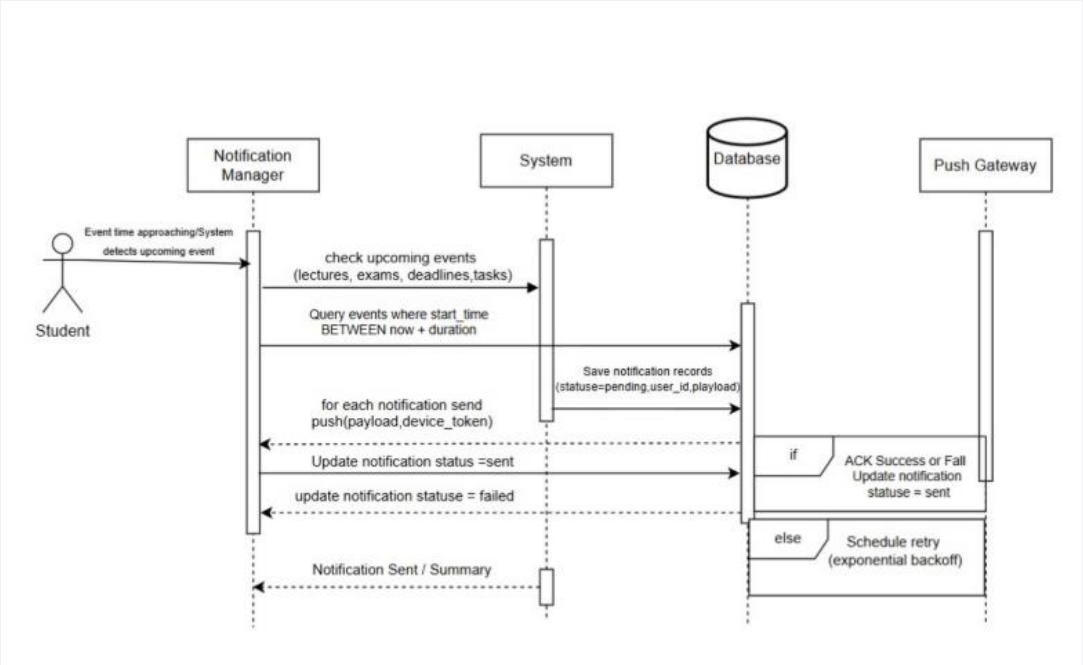


2-

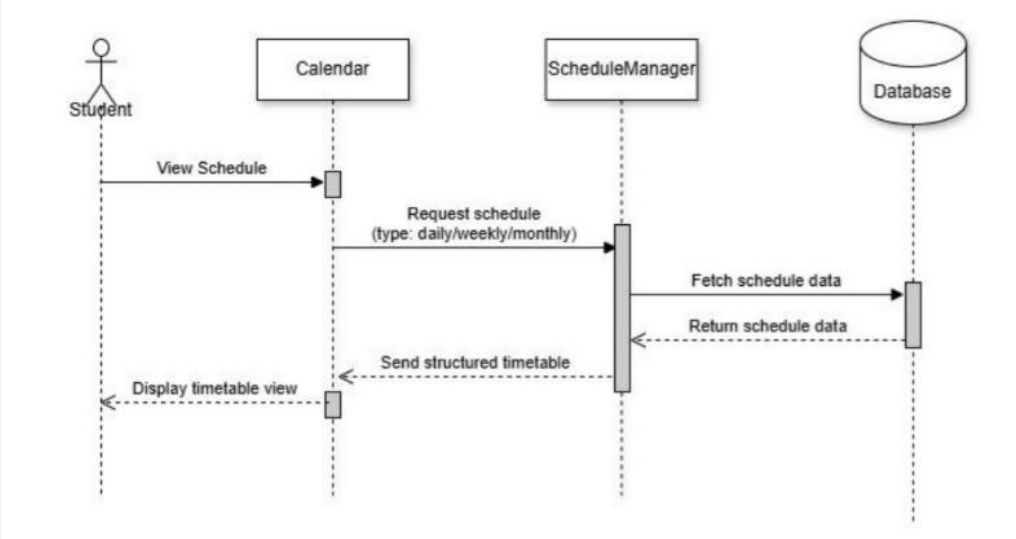


Sequence Diagram:

1-



2-



Sprint Meeting

project Name: Your Desk

Project Members: Roya – Layal – Lama – Albatol – Sara

Sprint: 2 Stand-up Meeting (8/11/2025)

Sprint Duration: 1 week

Scrum Master: Dr. Malak

Client: Roya – Layal – Lama – Albatol – Sara

Pair Programmers: Roya – Layal – Lama – Albatol – Sara

Component Name	**Story Sequence Number**	**Use Cases (e.g., functionalities)**
Schedule Manager	003	Students can view their schedules in a structured timetable (daily, weekly, or monthly).
Notification Manager	004	Students receive instant notifications for upcoming lectures, exams, deadlines, and tasks.

Follow-up Meeting Questions:

1. **What has been completed since the last meeting?**After finishing Sprint 1, we made some improvements to the task and course management features, then started working on the new functionalities of Sprint After that we began implementing the schedule display structure and the initial setup for instant notifications.
2. **What are you going to be working on next?**We will continue developing the schedule display feature by adding the monthly view, and we will begin integrating the instant notification system to remind users about upcoming lectures, exams, and deadlines.
3. **Do you have any issues/impediments?** Yes, we faced some issues synchronizing the timetable data with stored events and ensuring that instant notifications are triggered accurately and on time.

Scrum Master's comments based on the above questions:

Sprint Meeting

project Name: Your Desk

Project Members: Roya – Layal – Lama – Albatol – Sara

Sprint: 2 Stand-up Meeting (12/11/2025)

Sprint Duration: 1 week

Scrum Master: Dr. Malak

Client: Roya – Layal – Lama – Albatol – Sara

Pair Programmers: Roya – Layal – Lama – Albatol – Sara

Component Name	**Story Sequence Number**	**Use Cases (e.g., functionalities)**
Schedule Manager	003	Students can view their schedules in a structured timetable (daily, weekly, or monthly).
Notification Manager	004	Students receive instant notifications for upcoming lectures, exams, deadlines, and tasks.

Follow-up Meeting Questions:

1. **What has been completed since the last meeting?**We successfully implemented both schedule display (daily/weekly/monthly timetable) and instant notifications (real-time alerts for upcoming lectures, exams, deadlines, and tasks).
2. **What are you going to be working on next?**We will focus on writing the test cases for Sprint 2.
3. **Do you have any issues/impediments?**No, the system is working fine and the new functionalities are completed.

Scrum's Master commentsbased on the above questions:

Sprint Test Cases

Project Name: Your Desk

Project Members: Roya – Layal – Lama – Albatol – Sara

Sprint 2 Test Cases [14/11/2025]

- **Test Case Name:** [Sprint 2 – Schedule Manager]
- **Test Case Id:** [Your Desk – Schedule Manager]

TestCase No	Test Case Description	Expected Results	Outcome (Pass/ Fail/Other)
TC001	User navigates to the Calendar tab and selects a day that has both a course and a task scheduled (e.g., Nov 13, 2025).	The Calendar should show an indicator dot below the selected date. The list below the calendar should correctly display the details of all scheduled courses and tasks for that specific date.	pass
TC002	User selects a day with multiple courses and multiple tasks (e.g., Nov 12, 2025).	The courses and tasks should all appear correctly and in chronological order (by start time/due time) under the date header without any overlap.	pass
TC003	User selects a day that has no scheduled courses or tasks.	The Calendar should not show an indicator dot, and the area below the calendar should show a message or remain empty, indicating no scheduled items for that date	pass

TC004	User scrolls through the Calendar view to check the appearance of indicator dots on different days.	All days that have a scheduled item course or task) should display a clear indicator dot, allowing the user to easily identify busy days.	pass
TC005	A course reaches its end time while the user is viewing the Calendar.	The course should automatically be marked as completed (shown with a checkmark). The completed state must remain visible when navigating away and returning.	pass

Sprint Test Cases

Test Case Name:[Sprint 2 – Notification Manager]

Test Case Id: [YourDesk – Notification Manager]

TestCase No	Test Case Description	Expected Results	Outcome (Pass/ Fail/Other)
TC001	User adds a Course with a start time of 1:00 PM (Thursday) and the app is running in the background.	The system should successfully schedule a recurring weekly notification that fires 2 minutes before the course start time at 12:58 PM with the message "Starts in 2 minutes". It should also send a notification at the exact start time at 1:00 PM and another notification when the class ends at the specified end time.	pass
TC002	User adds a Task with a due date/time of 11:00 AM, and the app is in the background.	The system should successfully schedule two exact local notifications: 1) The first notification fires 1 hour before (10:00 AM) with the message: "Your deadline is in 1 hour". 2) The second notification fires exactly at the due time (11:00 AM) with the message: "Your task is due now".	pass

TC003	User modifies an existing Course/Task time/date.	The system should cancel the old scheduled notifications and reschedule new notifications at the updated times.	pass
TC004	User deletes an existing Course or Task.	The system should successfully cancel all associated scheduled local notifications to prevent unwanted notifications from firing.	pass
TC005	User opens the notification dropdown and views the course and task reminders.	All reminders should appear under their designated channel and display the correct course or task name.	pass

Sprint three

project Initiation

Project Description :In Sprint 3 of our project *YourDesk*, we focused on enhancing the user interface and improving the overall user experience, we added one function:

- **Theme Mode Selection (FR7)**

This function allows students to switch between dark mode and light mode based on their preference. By enabling theme customization, the system now provides a more comfortable viewing experience. The selected mode applies consistently across all screens, buttons, icons, and text, ensuring a unified and user-friendly interface.

With this update, the application becomes more adaptable and customizable, supporting students in using the app comfortably in various environments.

Prioritized Functional requirements:

Functional Requirements:

- FR7 (Low): The system shall provide dark mode and light mode theme selection.

Story Backlog

Component Name: Theme Mode Selection

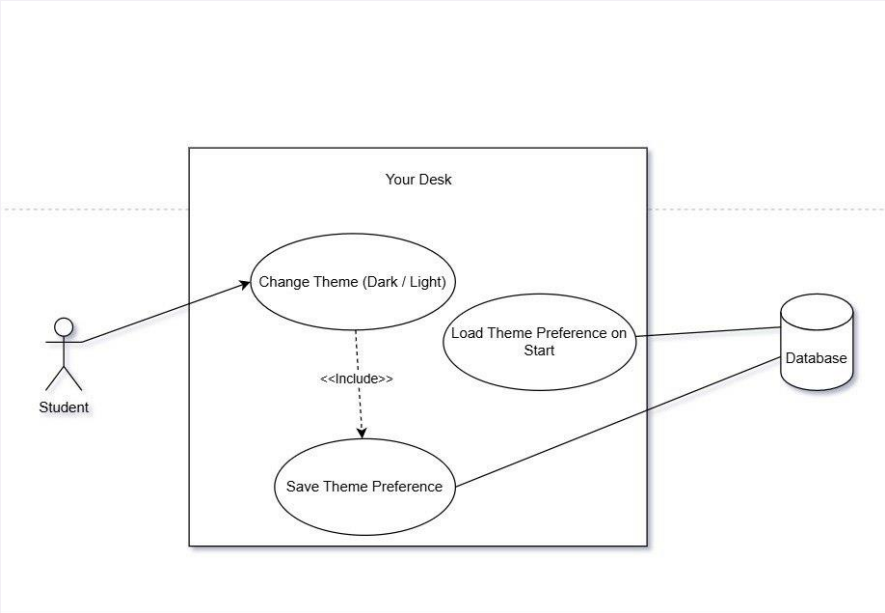
Story Name: Theme Mode Selection

Story Sequence No: 004

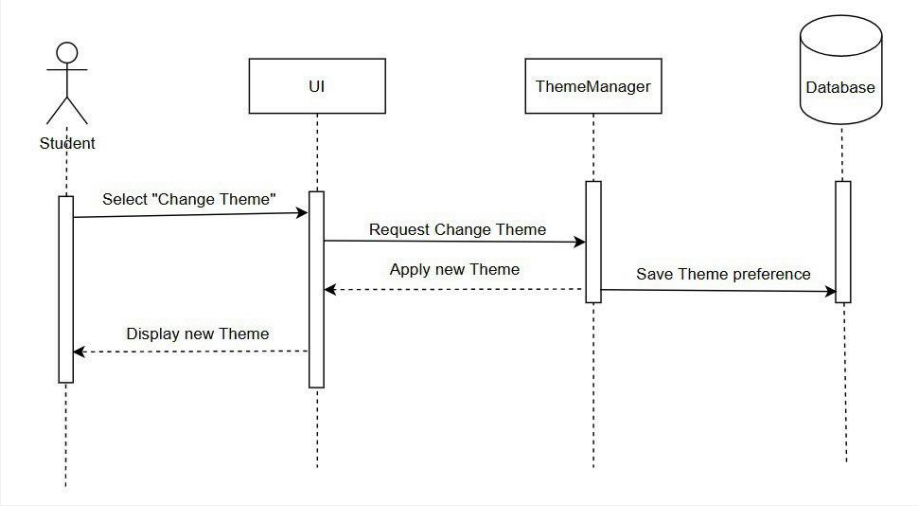
Story Short Description: Students can switch between dark mode and light mode to customize the application's appearance.

Story Long Description: Students shall be able to choose between dark mode and light mode through the screens. The system shall apply the selected theme across all screens, components, buttons, and text to maintain consistency. The interface must automatically adapt colors, backgrounds, and icons to ensure readability and usability in both modes. The selection should remain saved and persist even after closing or reopening the application.

Use Case Diagram:



Sequence Diagram:



Sprint Test Cases

Project Name: Your Desk

Project Members: Roya – Layal – Lama – Albatol – Sara

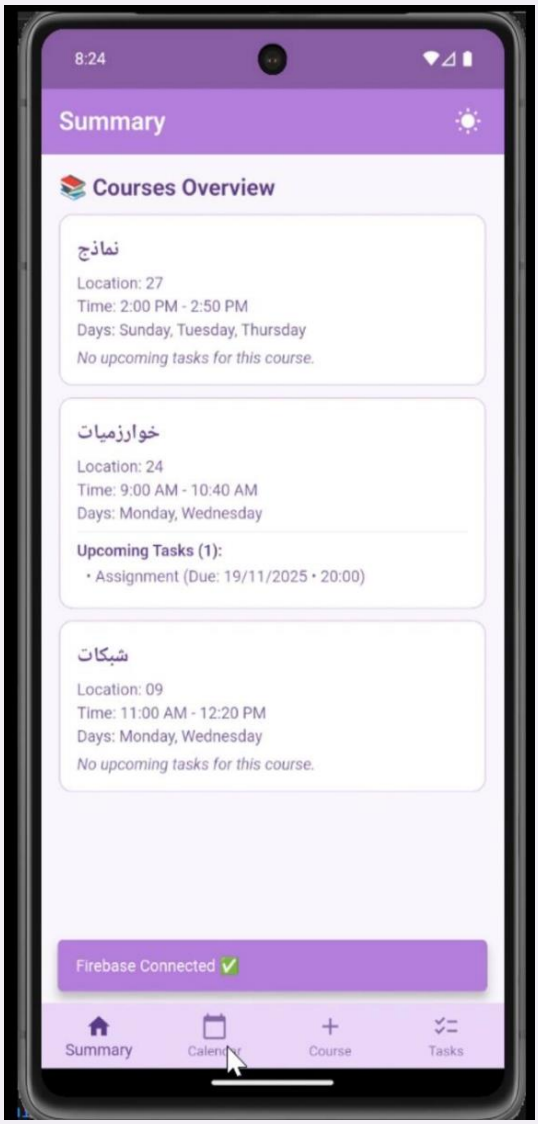
Sprint 3 Test Cases [1/12/2025]

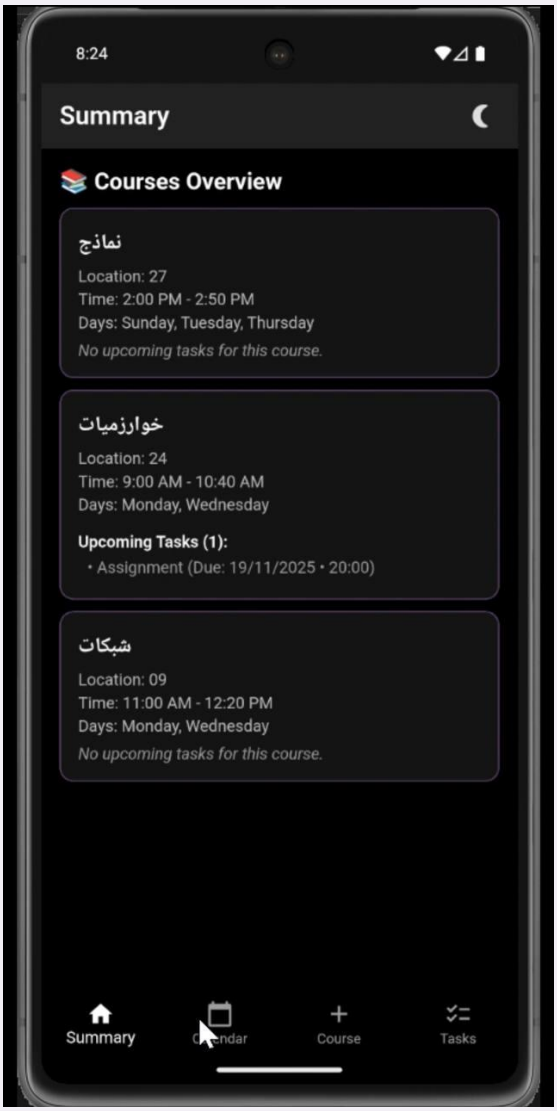
- **Test Case Name:** [Sprint 3– Theme Mode Selection]
- **Test Case Id:** [Your Desk – Theme Mode Selection]

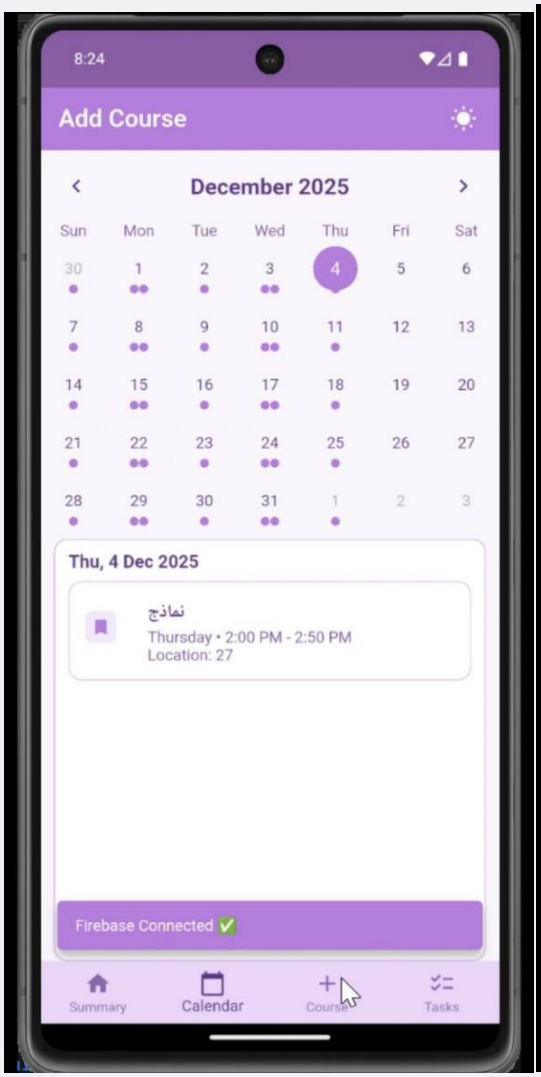
TestCase No	Test Case Description	Expected Results	Outcome (Pass/ Fail/Other)
TC001	User opens the app for the first time.	App should load in light mode by default and display a consistent light theme across all screens.	pass
TC002	While in light mode, user taps the theme icon in the corner of the screen.	Theme should immediately switch to dark mode on the current screen.	pass
TC003	While in dark mode, user taps the theme icon again.	Theme should switch back to light mode with all elements updated.	pass
TC004	User switches to dark mode then navigates between screens.	All screens should appear in dark mode consistently.	pass
TC005	User switches to light mode then navigates between screens.	All screens should appear in light mode consistently.	pass

TC006	User switches to dark mode, closes and reopens the app	App should reopen in dark mode.	pass
TC007	User switches to light mode, closes and reopens the app.	App should reopen in light mode.	pass
TC008	User checks that the theme icon appears on every main screen.	Theme icon should be visible and tappable on all main screens.	pass
TC009	User rapidly taps the theme icon multiple times.	System toggles correctly with no crashes or glitches.	pass

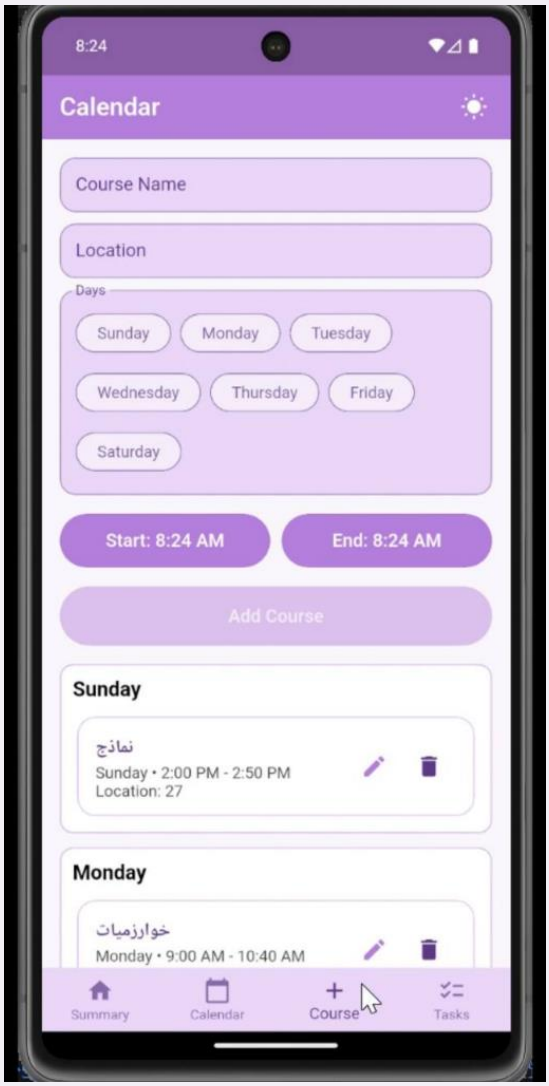
App Screenshots

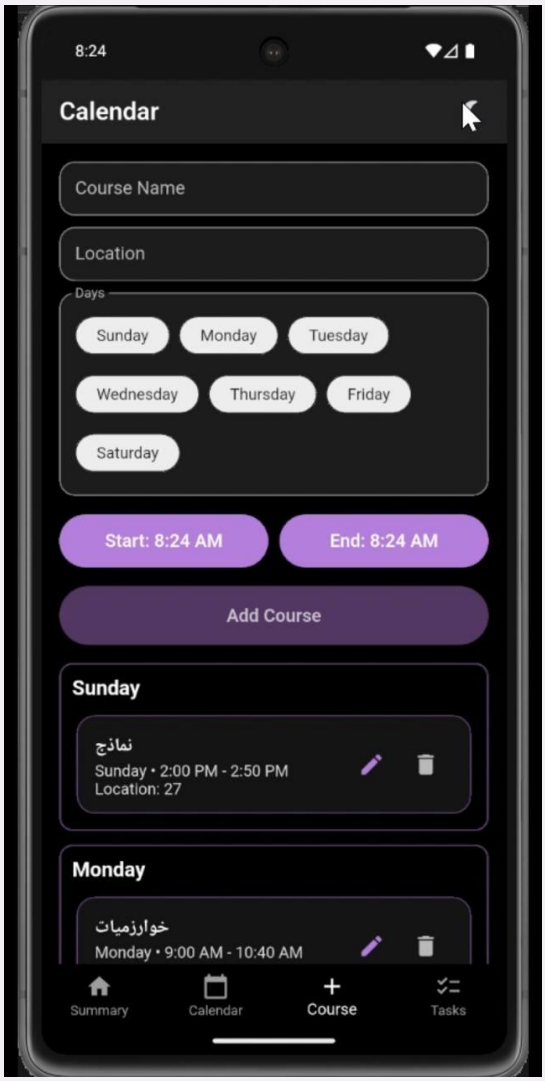

8:24
Summary
Courses Overview
نماذج
Location: 27
Time: 2:00 PM - 2:50 PM
Days: Sunday, Tuesday, Thursday
No upcoming tasks for this course.
خوارزميات
Location: 24
Time: 9:00 AM - 10:40 AM
Days: Monday, Wednesday
Upcoming Tasks (1):
• Assignment (Due: 19/11/2025 • 20:00)
شبكات
Location: 09
Time: 11:00 AM - 12:20 PM
Days: Monday, Wednesday
No upcoming tasks for this course.
Firebase Connected ✅
Summary
Course
Tasks

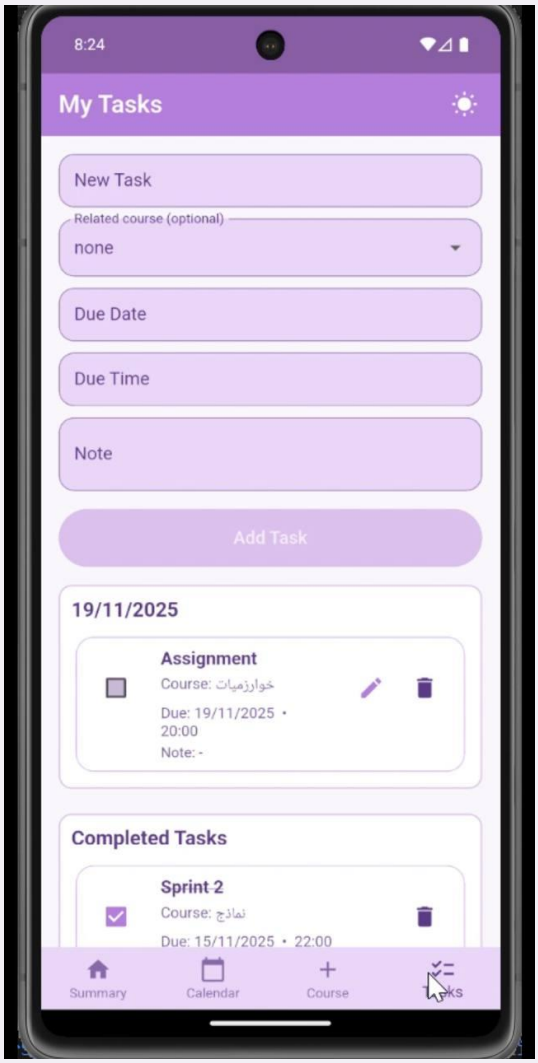

8:24
Summary
Courses Overview
نماذج
Location: 27
Time: 2:00 PM - 2:50 PM
Days: Sunday, Tuesday, Thursday
No upcoming tasks for this course.
خوارزميات
Location: 24
Time: 9:00 AM - 10:40 AM
Days: Monday, Wednesday
Upcoming Tasks (1):
• Assignment (Due: 19/11/2025 • 20:00)
شبكات
Location: 09
Time: 11:00 AM - 12:20 PM
Days: Monday, Wednesday
No upcoming tasks for this course.
Summary
Course
Tasks

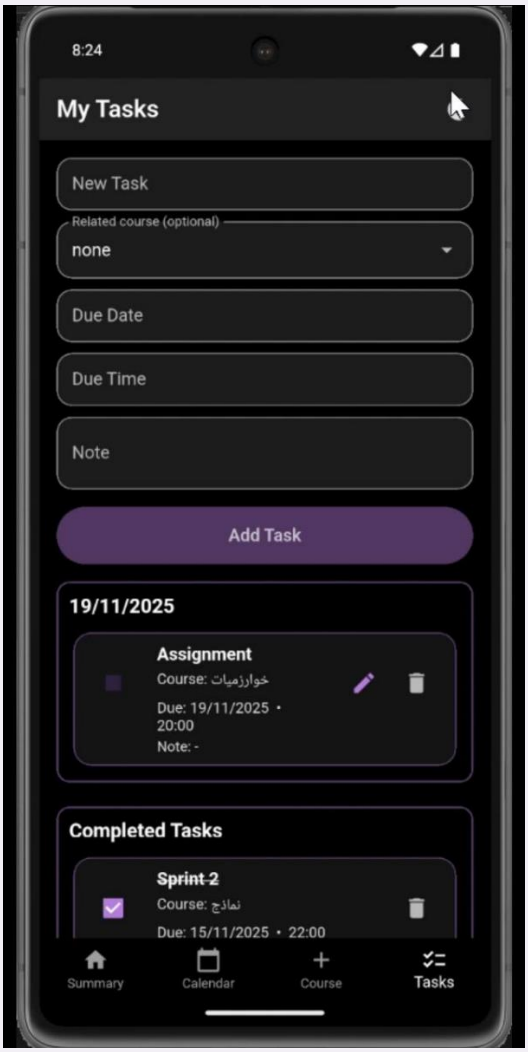

8:24
Add Course
December 2025
Sun Mon Tue Wed Thu Fri Sat
Thu, 4 Dec 2025
نماذج
Thursday • 2:00 PM - 2:50 PM
Location: 27
Firebase Connected ✅
Summary
Calendar
Course
Tasks

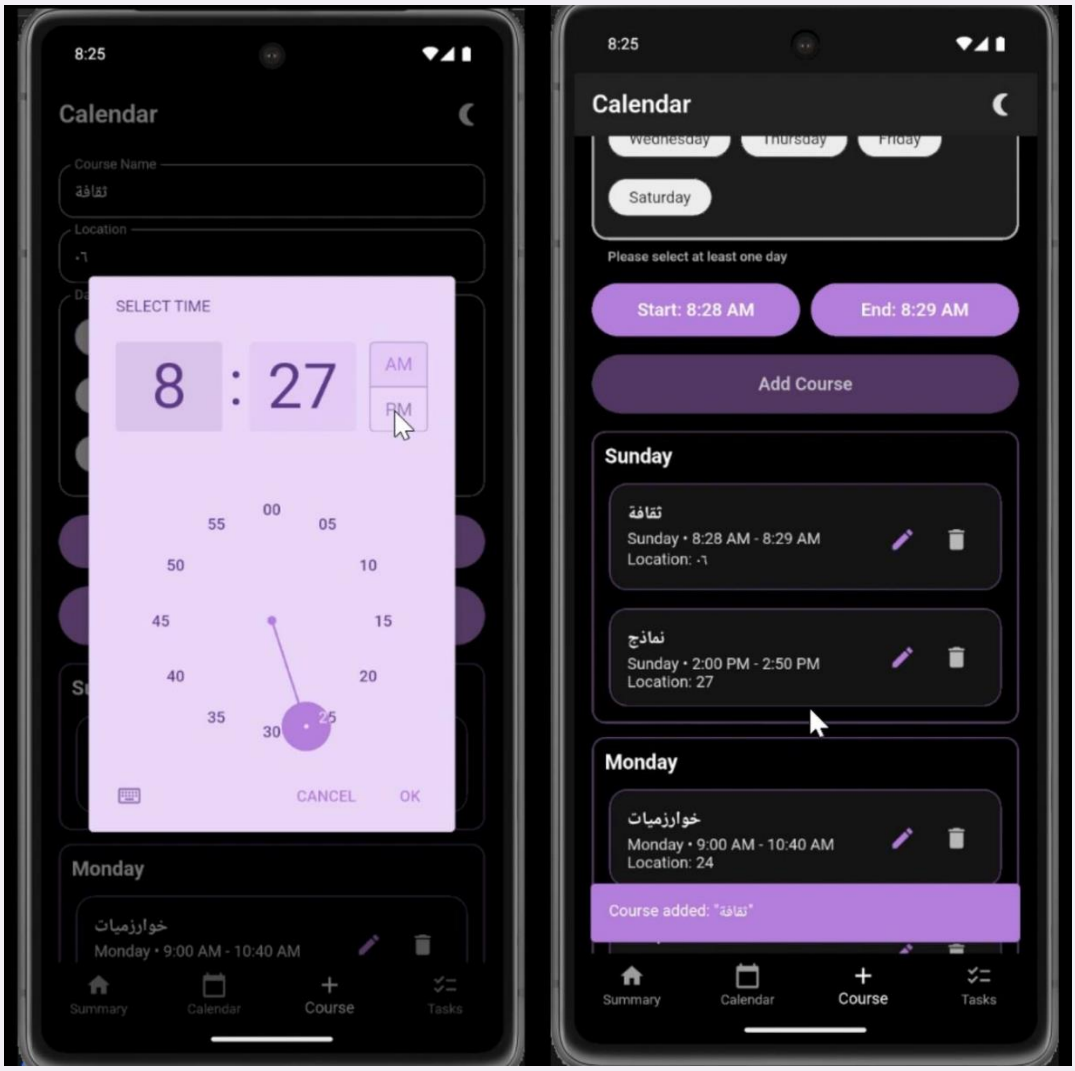
8:24
Add Course
December 2025
Sun Mon Tue Wed Thu Fri Sat
Thu, 4 Dec 2025
نماذج
Thursday • 2:00 PM - 2:50 PM
Location: 27
Summary
Calendar
Course
Tasks

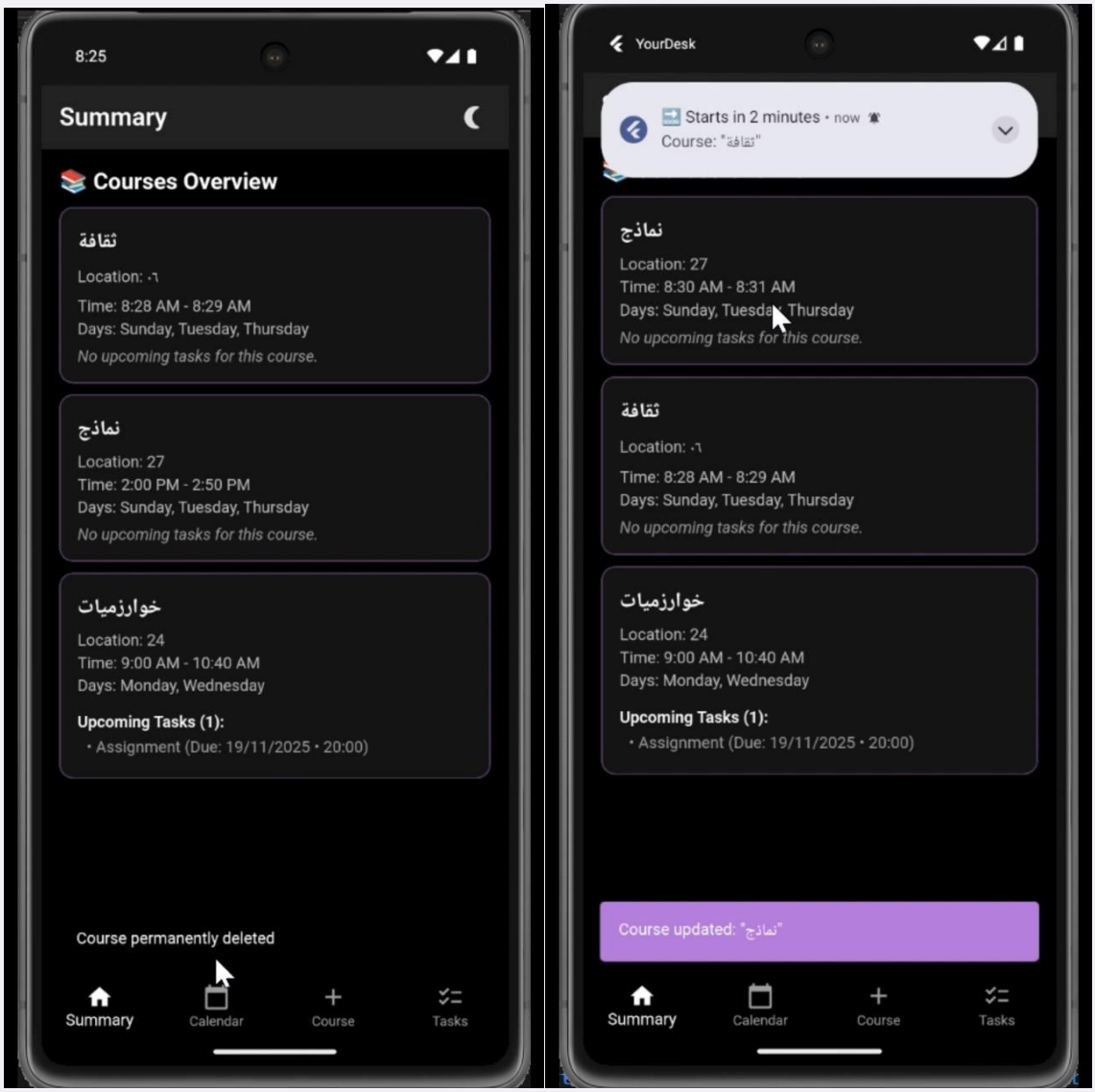

8:24
Calendar
Course Name
Location
Days
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Start: 8:24 AM
End: 8:24 AM
Add Course
Sunday
نماذج
Sunday • 2:00 PM - 2:50 PM
Location: 27
Monday
خوارزميات
Monday • 9:00 AM - 10:40 AM
Summary
Calendar
Course
Tasks

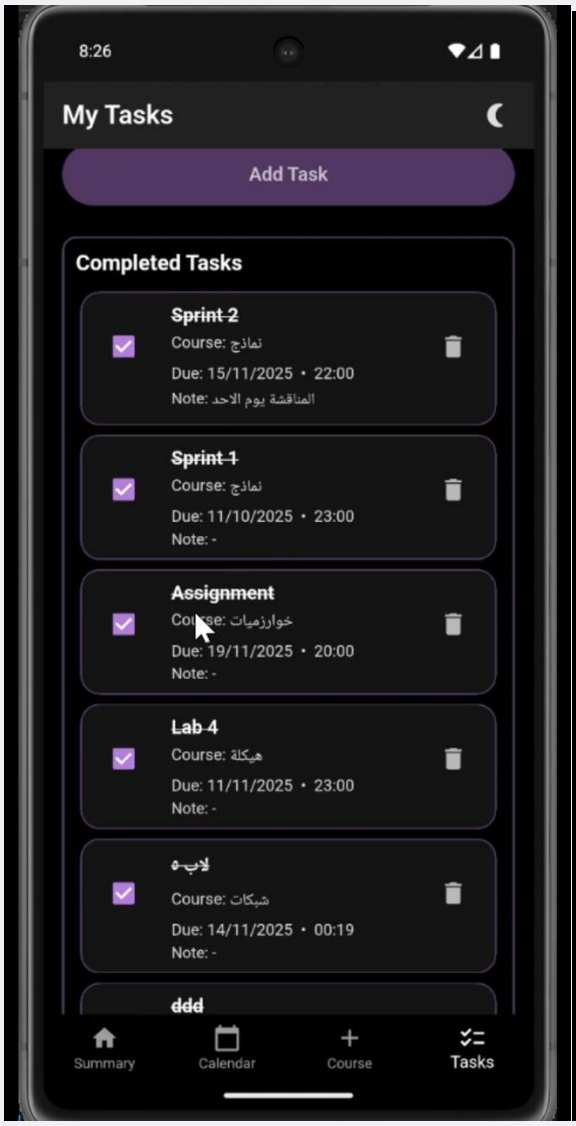

8:24
Calendar
Course Name
Location
Days
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Start: 8:24 AM
End: 8:24 AM
Add Course
Sunday
نماذج
Sunday • 2:00 PM - 2:50 PM
Location: 27
Monday
خوارزميات
Monday • 9:00 AM - 10:40 AM
Summary
Calendar
Course
Tasks

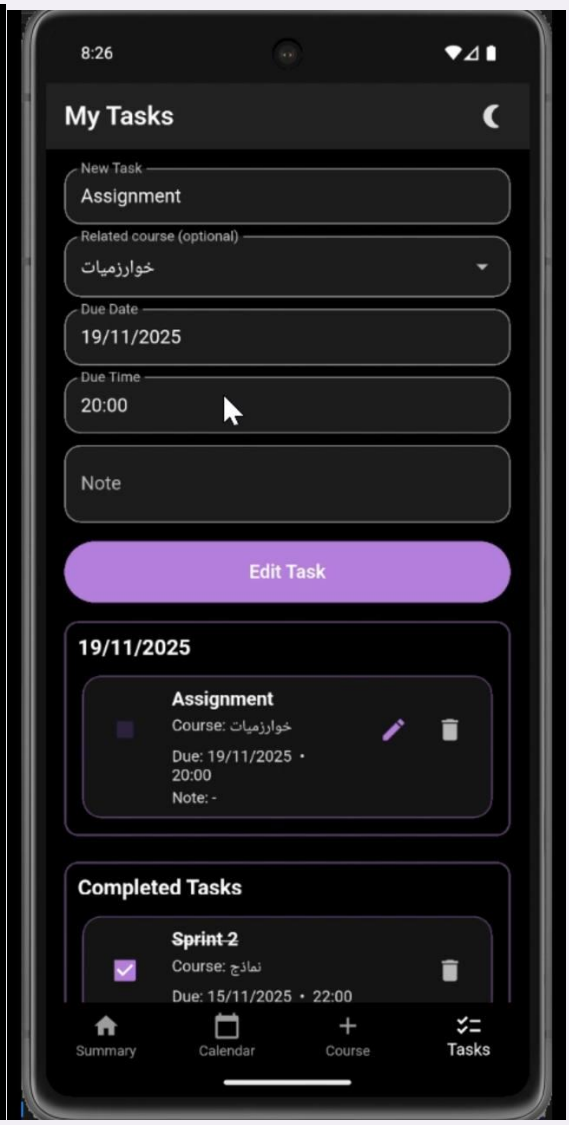

8:24
My Tasks
New Task
Related course (optional)
none
Due Date
Due Time
Note
Add Task
19/11/2025
Assignment
Course: خوارزميات
Due: 19/11/2025 • 20:00
Note: -
Completed Tasks
Sprint 2
Course: نماذج
Due: 15/11/2025 • 22:00
Summary
Calendar
Course

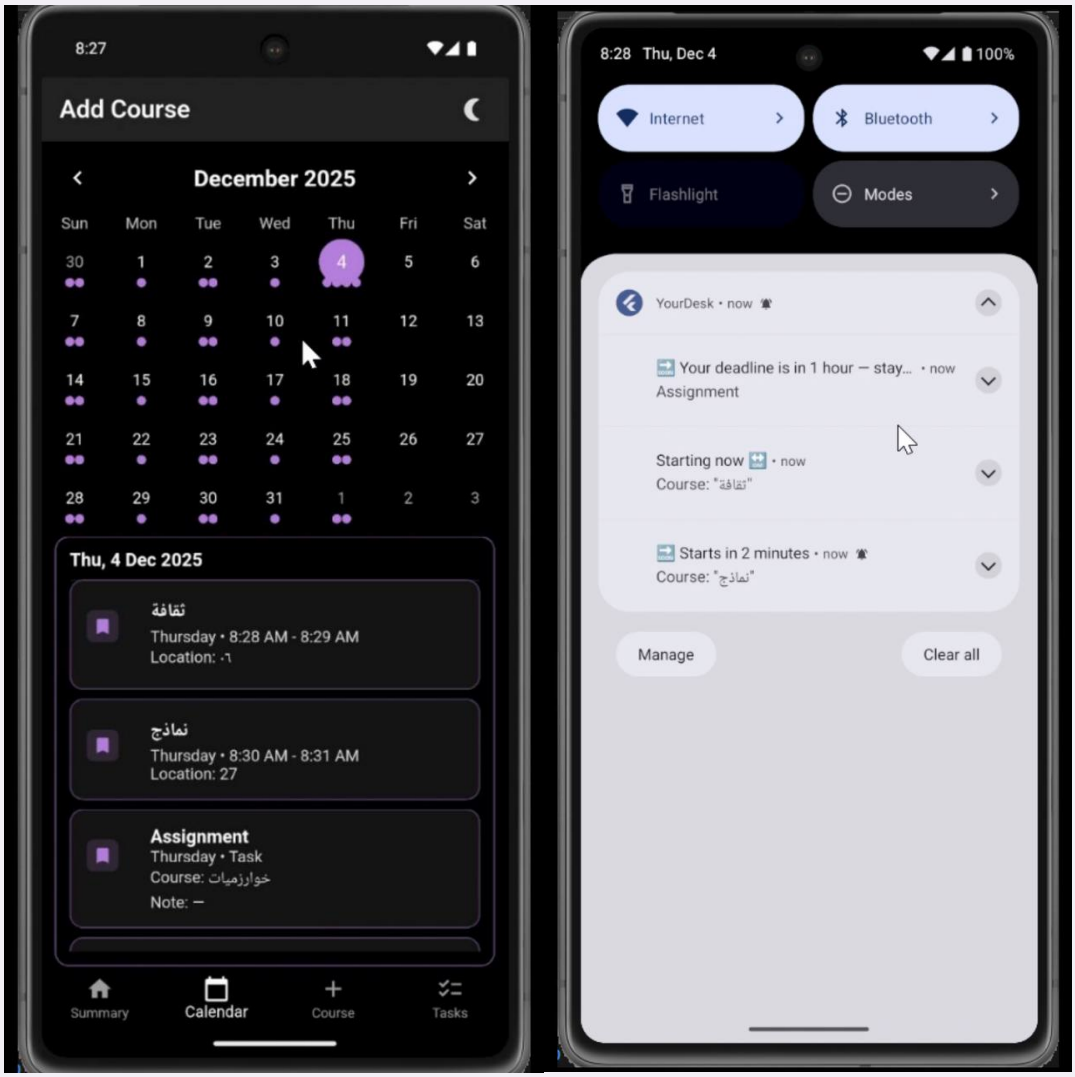

8:24
My Tasks
New Task
Related course (optional)
none
Due Date
Due Time
Note
Add Task
19/11/2025
Assignment
Course: خوارزميات
Due: 19/11/2025 • 20:00
Note: -
Completed Tasks
Sprint 2
Course: نماذج
Due: 15/11/2025 • 22:00
Summary
Calendar
Course
Tasks

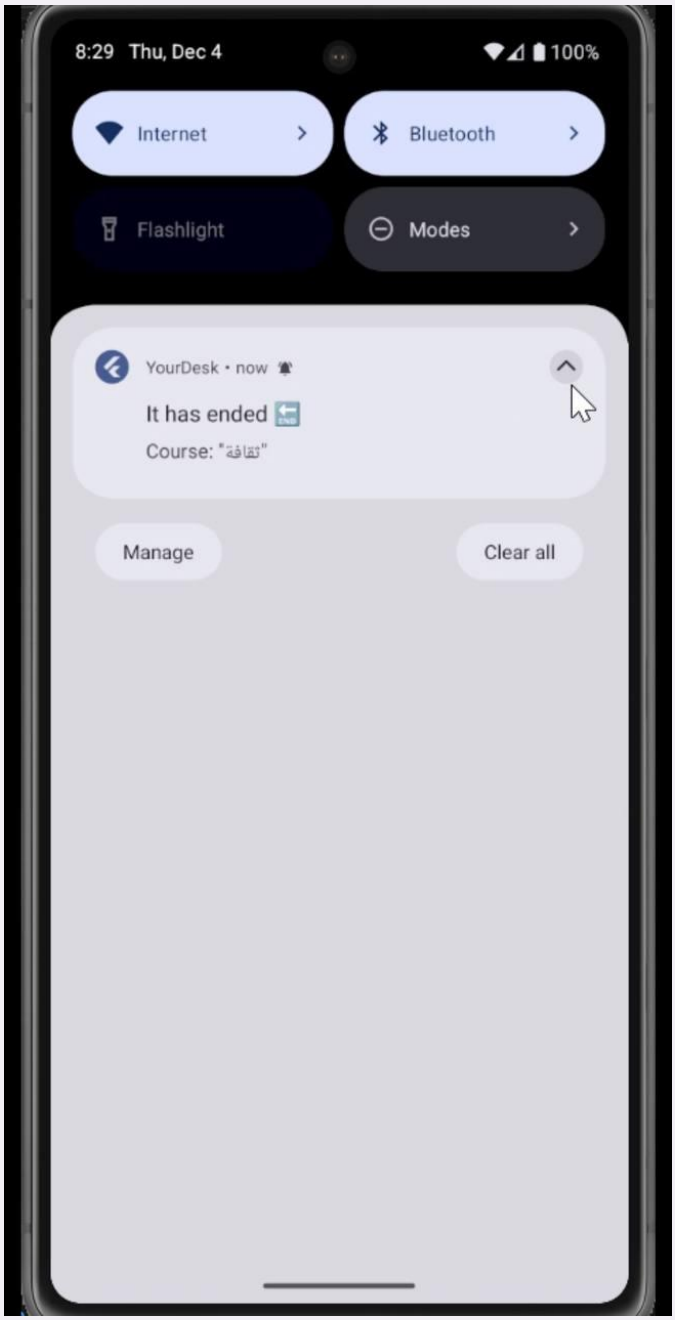

8:25
Calendar
Course Name
ثقافة
Location
SELECT TIME
8 : 27
AM
PM
CANCEL
OK
Monday
خوارزميات
Monday • 9:00 AM - 10:40 AM
Summary
Calendar
Course
Tasks
8:25
Calendar
Wednesday
Thursday
Friday
Saturday
Please select at least one day
Start: 8:28 AM
End: 8:29 AM
Add Course
Sunday
ثقافة
Sunday • 8:28 AM - 8:29 AM
Location: ٠٦
نماذج
Sunday • 2:00 PM - 2:50 PM
Location: 27
Monday
خوارزميات
Monday • 9:00 AM - 10:40 AM
Location: 24
Course added: "ثقافة"
Summary
Calendar
Course
Tasks


8:25
Summary
Courses Overview
ثقافة
Location: ٠٦
Time: 8:28 AM - 8:29 AM
Days: Sunday, Tuesday, Thursday
No upcoming tasks for this course.
نماذج
Location: 27
Time: 2:00 PM - 2:50 PM
Days: Sunday, Tuesday, Thursday
No upcoming tasks for this course.
خوارزميات
Location: 24
Time: 9:00 AM - 10:40 AM
Days: Monday, Wednesday
Upcoming Tasks (1):
• Assignment (Due: 19/11/2025 • 20:00)
Course permanently deleted
Summary
Calendar
Course
Tasks
YourDesk
Starts in 2 minutes • now
Course: "ثقافة"
نماذج
Location: 27
Time: 8:30 AM - 8:31 AM
Days: Sunday, Tuesday, Thursday
No upcoming tasks for this course.
ثقافة
Location: ٠٦
Time: 8:28 AM - 8:29 AM
Days: Sunday, Tuesday, Thursday
No upcoming tasks for this course.
خوارزميات
Location: 24
Time: 9:00 AM - 10:40 AM
Days: Monday, Wednesday
Upcoming Tasks (1):
• Assignment (Due: 19/11/2025 • 20:00)
Course updated: "نماذج"
Summary
Calendar
Course
Tasks


8:26
My Tasks
Add Task
Completed Tasks
Sprint 2
Course: نماذج
Due: 15/11/2025 • 22:00
Note: المناقشة يوم الاحد
Sprint 1
Course: نماذج
Due: 11/10/2025 • 23:00
Note: -
Assignment
Course: خوارزميات
Due: 19/11/2025 • 20:00
Note: -
Lab 4
Course: هيكلة
Due: 11/11/2025 • 23:00
Note: -
لاب ٥
Course: شبكات
Due: 14/11/2025 • 00:19
Note: -
ddd
Summary
Calendar
Course
Tasks


8:26
My Tasks
New Task
Assignment
Related course (optional)
خوارزميات
Due Date
19/11/2025
Due Time
20:00
Note
Edit Task
19/11/2025
Assignment
Course: خوارزميات
Due: 19/11/2025 • 20:00
Note: -
Completed Tasks
Sprint 2
Course: نماذج
Due: 15/11/2025 • 22:00
Summary
Calendar
Course
Tasks


8:27
Add Course
December 2025
Sun Mon Tue Wed Thu Fri Sat
Thu, 4 Dec 2025
ثقافة
Thursday • 8:28 AM - 8:29 AM
Location: ٠٦
نماذج
Thursday • 8:30 AM - 8:31 AM
Location: 27
Assignment
Thursday • Task
Course: خوارزميات
Note: –
Summary
Calendar
Course
Tasks
8:28 Thu, Dec 4
100%
Internet
Bluetooth
Flashlight
Modes
YourDesk • now
Your deadline is in 1 hour – stay... • now
Assignment
Starting now • now
Course: "ثقافة"
Starts in 2 minutes • now
Course: "نماذج"
Manage
Clear all


8:29 Thu, Dec 4
100%
Internet
Bluetooth
Flashlight
Modes
YourDesk • now
It has ended
Course: "ثقافة"
Manage
Clear all

Code

Main:

```dart
import 'dart:io';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:timezone/data/latest.dart' as tz;
import 'package:timezone/timezone.dart' as tzl;

import 'screens/splash_page.dart';
import 'screens/home_shell.dart';
import 'screens/ColorsManager.dart';
import 'theme_controller.dart';

final FlutterLocalNotificationsPlugin courseLocalNotificationsPlugin =
    FlutterLocalNotificationsPlugin();

const AndroidNotificationChannel _coursesChannel = AndroidNotificationChannel(
  'courses_channel',
  'Courses',
  description: 'Course reminders',
  importance: Importance.max,
  enableVibration: true,
  showBadge: true,
  playSound: true,
);

Future<void> _createCoursesChannel() async {
  await courseLocalNotificationsPlugin
      .resolvePlatformSpecificImplementation<
        AndroidFlutterLocalNotificationsPlugin
      >()
      ?.createNotificationChannel(_coursesChannel);
}

Future<void> _initCourseLocalNotifications() async {
```

```dart
}

const MethodChannel _sysChannel = MethodChannel('exact_alarm_check');

Future<bool> _canScheduleExactAlarms() async {
  if (!Platform.isAndroid) return true;
  try {
    final ok = await _sysChannel.invokeMethod<bool>('canScheduleExactAlarms');
    return ok ?? false;
  } catch (_) {
    return false;
  }
}

Future<void> _openExactAlarmSettings() async {
  if (!Platform.isAndroid) return;
  try {
    await _sysChannel.invokeMethod('openExactAlarmSettings');
  } catch (_) {
    try {
      await _sysChannel.invokeMethod('openExactAlarmSettings');
    } catch (_) {}
  }
}

Future<void> _openBatteryOptimizationSettings() async {
  if (!Platform.isAndroid) return;
  try {
    await _sysChannel.invokeMethod('openBatteryOptimizationSettings');
  } catch (_) {}
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
```

Sammary_page:

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'ColorsManager.dart';
import '../firebase/firestore_service.dart';
import '../models/course.dart';

class SummaryPage extends StatefulWidget {
  const SummaryPage({super.key});

  @override
  State<SummaryPage> createState() => _SummaryPageState();
}

class _SummaryPageState extends State<SummaryPage> {
  List<Course> _courses = [];
  List<Map<String, dynamic>> _tasks = [];
  final _tasksCol = FirebaseFirestore.instance.collection('tasks');
  StreamSubscription<QuerySnapshot<Map<String, dynamic>>>? _tasksSub;
  StreamSubscription<List<Course>>? _coursesSub;

  @override
  void initState() {
    super.initState();
    _subscribeCourses();
    _subscribeTasks();
  }

  @override
  void dispose() {
    _tasksSub?.cancel();
    _coursesSub?.cancel();
    super.dispose();
  }

  void _subscribeCourses() {
```

```
class _SummaryPageState extends State<SummaryPage> {
  void _subscribeCourses() {
    });
  }

  void _subscribeTasks() {
    _tasksSub = _tasksCol
        .orderBy('dueDateTime', descending: false)
        .snapshots()
        .listen((snap) {
      final list = <Map<String, dynamic>>[];
      for (final d in snap.docs) {
        final data = d.data();
        list.add({
          'id': d.id,
          'task': (data['task'] ?? '').toString(),
          'date': (data['date'] ?? '').toString(),
          'time': (data['time'] ?? '').toString(),
          'courseTitle': data['courseTitle']?.toString(),
          'isCompleted': (data['isCompleted'] ?? false) as bool,
          'dueDateTime': data['dueDateTime'] as Timestamp?,
        });
      }
      if (!mounted) return;
      setState(() {
        _tasks = list
            .where((t) => !(t['isCompleted'] as bool))
            .toList();
      });
    });
  }

  String _fmt12(int h, int m) {
    final t = TimeOfDay(hour: h, minute: m);
    final h12 = t.hourOfPeriod == 0 ? 12 : t.hourOfPeriod;
```

Splash_page:

```
import 'dart:math' as math;
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'ColorsManager.dart';

class SplashPage extends StatefulWidget {
  const SplashPage({super.key});

  @override
  State<SplashPage> createState() => _SplashPageState();
}

class _SplashPageState extends State<SplashPage>
    with SingleTickerProviderStateMixin {
  late final AnimationController _ctrl;

  @override
  void initState() {
    super.initState();
    _ctrl = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 10),
    )..repeat();
  }

  @override
  void dispose() {
    _ctrl.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;

    final mauveLight = ColorsManager.mauveLight;
```

```
final snowWhite = ColorsManager.snowWhite;

final bgTop = isDark ? const Color(0xFF050509) : snowWhite;
final bgBottom = isDark ? const Color(0xFF14111C) : mauveLight;

final textMainColor = isDark ? Colors.white : mauveDark;
final textSubColor = isDark ? Colors.white70 : mauveDark.withOpacity(.75);

return Scaffold(
  body: AnimatedBuilder(
    animation: _ctrl,
    builder: (context, _) {
      return Container(
        decoration: BoxDecoration(
          gradient: LinearGradient(
            colors: [bgTop, bgBottom],
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
          ),
        ),
        child: Stack(
          children: [
            _FloatingBlob(
              ctrl: _ctrl,
              size: 220,
              x: .15,
              y: .25,
              opacity: isDark ? .14 : .18,
            ),
            _FloatingBlob(
              ctrl: _ctrl,
              size: 160,
              x: .85,
```

Course_Calendar_Page:

```dart
import 'dart:async';

import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:table_calendar/table_calendar.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

import 'ColorsManager.dart';
import '../../models/course.dart';
import '../../firebase/firestore_service.dart';

class CourseCalendarPage extends StatefulWidget {
  const CourseCalendarPage({super.key});

  @override
  State<CourseCalendarPage> createState() => _CourseCalendarPageState();
}

class _CourseCalendarPageState extends State<CourseCalendarPage> {
  List<Course> _courses = [];
  final List<Map<String, dynamic>> _tasks = [];

  // Firestore tasks stream
  final _tasksCol = FirebaseFirestore.instance.collection('tasks');
  StreamSubscription<QuerySnapshot<Map<String, dynamic>>>? _tasksSub;
  DateTime _focusedDay = DateTime.now();
  DateTime? _selectedDay;

  @override
  void initState() {
    super.initState();
    _subscribeTasks();
    _subscribeToCourses();
    _loadTasks();
    _selectedDay = DateTime.now();
```

```dart
class _CourseCalendarPageState extends State<CourseCalendarPage> {
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: [
            ),
          ),
          daysOfWeekStyle: DaysOfWeekStyle(
            weekdayStyle: TextStyle(color: calendarTextColor.withOpacity(.8)),
            weekendStyle: TextStyle(color: calendarTextColor.withOpacity(.8)),
          ),
        ),

        const SizedBox(height: 8),

        Expanded(
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 12),
            child: events.isEmpty
                ? Center(
                    child: Text(
                      'No courses or tasks on this day',
                      style: TextStyle(color: secondaryTextColor),
                    )
                  )
                : Container(
                    decoration: BoxDecoration(
                      color: dayCardBg,
                      borderRadius: BorderRadius.circular(12),
                      border: Border.all(
                        color: ColorsManager.mauveMedium.withOpacity(.6),
                      )
                    ),
                    padding: const EdgeInsets.fromLTRB(12, 10, 12, 12),
                    child: Column(
```

Add_Course_Page:

```
import 'dart:convert';
import 'dart:io';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/material.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:permission_handler/permission_handler.dart';
import 'package:timezone/data/latest.dart' as tz;
import 'package:timezone/timezone.dart' as tz;

import 'ColorsManager.dart';

final FlutterLocalNotificationsPlugin courseLocalNotificationsPlugin =
    FlutterLocalNotificationsPlugin();
bool _courseTzInitialized = false;

Future<void> _createNotificationChannel() async {
  const AndroidNotificationChannel channel = AndroidNotificationChannel(
    'courses_channel',
    'Course Notifications',
    description: 'Notifications for course reminders and schedules',
    importance: Importance.max,
    enableVibration: true,
    playSound: true,
    showBadge: true,
  );
  await courseLocalNotificationsPlugin
      .resolvePlatformSpecificImplementation<
        AndroidFlutterLocalNotificationsPlugin
      >()
      ?.createNotificationChannel(channel);
}

Future<void> _initCourseLocalNotifications() async {
  if (!_courseTzInitialized) {
```

Restricted Mode ⊗ 0 ⚠ 0 Ln 240, Col 1

```
class AddCoursePage extends StatefulWidget {
  const AddCoursePage({super.key});

  @override
  State<AddCoursePage> createState() => _AddCoursePageState();
}

class _AddCoursePageState extends State<AddCoursePage> {
  final _formKey = GlobalKey<FormState>();

  final TextEditingController courseController = TextEditingController();
  final TextEditingController locationController = TextEditingController();

  TimeOfDay startTime = TimeOfDay.now();
  TimeOfDay endTime = TimeOfDay.now();

  int? editingIndex;
  bool _saving = false;

  Set<String> selectedDays = {};

  final List<String> daysOfWeek = const [
    'Sunday',
    'Monday',
    'Tuesday',
    'Wednesday',
    'Thursday',
    'Friday',
    'Saturday',
  ];

  List<Map<String, String>> courses = [];
  Map<String, List<Map<String, String>>> coursesByDay = {};
```

Tasks_Page:

```
class _TasksPageState extends State<TasksPage> {
  final List<Map<String, dynamic>> tasks = [];
  final TextEditingController taskController = TextEditingController();
  final TextEditingController dateController = TextEditingController();
  final TextEditingController timeController = TextEditingController();
  final TextEditingController noteController = TextEditingController();

  TimeOfDay? _selectedTime;

  List<String> _courseTitles = [];
  String? _selectedCourseTitle;

  String? selectedTask;
  int? selectedTaskIndex;
  Map<String, List<Map<String, dynamic>>> tasksByDate = {};

  final List<String> successMessages = const [
    "👏 Great job! Task completed!",
    "🔥 You did it! Keep going!",
    "🌟 Well done! Another step forward!",
    "💪 Proud of you! Keep shining!",
    "✅ One more task crushed!",
  ];
  final List<String> motivationMessages = const [
    "💡 Don't give up, you're almost there!",
    "🚀 Come on! You can finish this task!",
    "✨ Stay focused, you got this!",
    "⏳ Just a little push and you're done!",
    "🔥 Keep the momentum alive!",
  ];

  final _tasksCol = FirebaseFirestore.instance.collection('tasks');
  final _coursesCol = FirebaseFirestore.instance.collection('courses');
  final _remindersCol = FirebaseFirestore.instance.collection('reminders');

  StreamSubscription<QuerySnapshot<Map<String, dynamic>>>? _tasksSub;
```

```
import 'dart:async';

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/material.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:timezone/data/latest.dart' as tz;
import 'package:timezone/timezone.dart' as tz;

import 'ColorsManager.dart';

final FlutterLocalNotificationsPlugin taskLocalNotificationsPlugin =
    FlutterLocalNotificationsPlugin();
bool _taskTzInitialized = false;

Future<void> _initTaskLocalNotifications() async {
  if (!_taskTzInitialized) {
    tz.initializeTimeZones();
    tz.setLocalLocation(tz.getLocation('Asia/Riyadh'));
    _taskTzInitialized = true;
  }

  const androidInit = AndroidInitializationSettings('@mipmap/ic_launcher');
  const initSettings = InitializationSettings(android: androidInit);
  await taskLocalNotificationsPlugin.initialize(initSettings);

  const AndroidNotificationChannel channel = AndroidNotificationChannel(
    'tasks_channel',
    'Task Notifications',
    description: 'Task reminders',
    importance: Importance.max,
    enableVibration: true,
    playSound: true,
    showBadge: true,
  );
```

Course:

```dart
import 'package:flutter/material.dart';
import 'dart:convert';

class Course {
  final String? id;
  final String title;
  final String location;

  final List<String> days;
  final int startHour;
  final int startMinute;
  final int endHour;
  final int endMinute;

  Course({
    this.id,
    required this.title,
    required this.location,
    required this.days,
    required this.startHour,
    required this.startMinute,
    required this.endHour,
    required this.endMinute,
  });

  // ===== Helpers =====
  static int _toInt(dynamic v) => int.tryParse(v?.toString() ?? '') ?? 0;

  static int weekdayFromName(String name) {
    switch (name.toLowerCase()) {
      case 'monday':
        return DateTime.monday;
      case 'tuesday':
        return DateTime.tuesday;
      case 'wednesday':
        return DateTime.wednesday;
```

```dart

  bool occursOnWeekday(int weekday) =>
      days.any((d) => weekdayFromName(d) == weekday);

  String intervalLabel(BuildContext context) {
    final s = TimeOfDay(hour: startHour, minute: startMinute).format(context);
    final e = TimeOfDay(hour: endHour, minute: endMinute).format(context);
    return '$s - $e';
  }

  String get daysLabel => days.join(', ');

  // ===== JSON =====
  factory Course.fromJson(Map<String, dynamic> json) {

    List<String> parsedDays = [];
    final rawDays = json['days'];

    if (rawDays != null) {
      if (rawDays is List) {
        parsedDays = rawDays.map((e) => e.toString()).toList();
      } else if (rawDays is String) {

        try {
          final arr = (jsonDecode(rawDays) as List).map((e) => e.toString()).toList();
          parsedDays = arr;
        } catch (_) {
          parsedDays = [rawDays];
        }
      }
    }

    if (parsedDays.isEmpty) {
      final legacyDay = (json['day'] as String?)?.trim();
      if (legacyDay != null && legacyDay.isNotEmpty) parsedDays = [legacyDay];
```

Class_Item:

```dart
import 'package:flutter/material.dart';
import 'dart:convert';

class ClassItem {
  final String title;
  final String type; // Lecture/Lab/Online
  final String location;
  final TimeOfDay start;
  final TimeOfDay end;
  final DateTime day;

  // Constructor
  ClassItem({
    required this.title,
    required this.type,
    required this.location,
    required this.start,
    required this.end,
    required this.day,
  });


  Map<String, dynamic> toMap() => {
        'title': title,
        'type': type,
        'location': location,
        'startH': start.hour,
        'startM': start.minute,
        'endH': end.hour,
        'endM': end.minute,
        'day': day.toIso8601String(),
      };


  factory ClassItem.fromMap(Map<String, dynamic> m) {
```

```dart
        'endM': end.minute,
        'day': day.toIso8601String(),
      };


  factory ClassItem.fromMap(Map<String, dynamic> m) {

    return ClassItem(
      title: m['title'] ?? '',
      type: m['type'] ?? '',
      location: m['location'] ?? '',
      start: TimeOfDay(
        hour: int.tryParse(m['startH'].toString()) ?? 0,
        minute: int.tryParse(m['startM'].toString()) ?? 0,
      ),
      end: TimeOfDay(
        hour: int.tryParse(m['endH'].toString()) ?? 0,
        minute: int.tryParse(m['endM'].toString()) ?? 0,
      ),
      day: DateTime.parse(m['day']),
    );
  }


  String toJson() => json.encode(toMap());


  factory ClassItem.fromJson(String s) {
    return ClassItem.fromMap(json.decode(s));
  }
}
```

Home_Shell:

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

import 'add_course_page.dart';
import 'course_calendar_page.dart';
import 'tasks_page.dart';
import 'summary_page.dart';
import '../theme_controller.dart';

class HomeShell extends StatefulWidget {
  const HomeShell({super.key});

  @override
  State<HomeShell> createState() => _HomeShellState();
}

class _HomeShellState extends State<HomeShell> {
  int _selectedIndex = 0;

  final List<Widget> _pages = const [
    SummaryPage(),
    CourseCalendarPage(),
    AddCoursePage(),
    TasksPage(),
  ];

  final List<String> _titles = const [
    'Summary',
    'Add Course',
    'Calendar',
    'My Tasks',
  ];

```

```dart
class _HomeShellState extends State<HomeShell> {
  Future<void> _loadBootTab() async {
    if (boot != null && boot >= 0 && boot < _pages.length) {
      setState(() => _selectedIndex = boot);
      await prefs.remove('boot_tab');
    }
  }

  void _onItemTapped(int index) => setState(() => _selectedIndex = index);

  void _notifyIfFirebaseReady() {
    WidgetsBinding.instance.addPostFrameCallback((_) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(const SnackBar(content: Text('Firebase Connected ✅')));
    });
  }

  Future<void> _setupPush() async {
    final messaging = FirebaseMessaging.instance;

    final settings = await messaging.requestPermission(
      alert: true,
      badge: true,
      sound: true,
    );
    debugPrint('🔔 Push permission: ${settings.authorizationStatus}');

    final token = await messaging.getToken();
    debugPrint('📱 FCM TOKEN: $token');

    if (token != null) {
      await FirebaseFirestore.instance
          .collection('deviceTokens')
          .doc(token)
          .set({
```

Theme_Controller:

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class AppThemeController {
  static const _key = 'app_theme_mode';

  static final ValueNotifier<ThemeMode> mode = ValueNotifier<ThemeMode>(
    ThemeMode.system,
  );

  static Future<void> init() async {
    final prefs = await SharedPreferences.getInstance();
    final saved = prefs.getString(_key);

    switch (saved) {
      case 'light':
        mode.value = ThemeMode.light;
        break;
      case 'dark':
        mode.value = ThemeMode.dark;
        break;
      default:
        mode.value = ThemeMode.system;
    }
  }

  static Future<void> _save(ThemeMode newMode) async {
    final prefs = await SharedPreferences.getInstance();
    switch (newMode) {
      case ThemeMode.light:
        await prefs.setString(_key, 'light');
        break;
      case ThemeMode.dark:
        await prefs.setString(_key, 'dark');
        break;
```

```dart
  static Future<void> _save(ThemeMode newMode) async {
    final prefs = await SharedPreferences.getInstance();
    switch (newMode) {
      case ThemeMode.light:
        await prefs.setString(_key, 'light');
        break;
      case ThemeMode.dark:
        await prefs.setString(_key, 'dark');
        break;
      case ThemeMode.system:
        await prefs.remove(_key);
        break;
    }
  }

  static void toggle() async {
    if (mode.value == ThemeMode.dark) {
      mode.value = ThemeMode.light;
      await _save(ThemeMode.light);
    } else {
      mode.value = ThemeMode.dark;
      await _save(ThemeMode.dark);
    }
  }

  static Future<void> set(ThemeMode newMode) async {
    mode.value = newMode;
    await _save(newMode);
  }
}
```

Colors_Manager:

```dart
import 'package:flutter/material.dart';

class ColorsManager {
  static const Color periwinkleBlue = Color(0xFFB4C4FE);
  static const Color softYellow = Color(0xFFFFF580);
  static const Color mintGreen = Color(0xFFD0F4EA);
  static const Color pinkBlush = Color(0xFFFFC0F5);
  static const Color iceBlue = Color(0xFFF4F7FF);
  static const Color royalBlue = Color(0xFF3B6BB9);
  static const Color snowWhite = Color(0xFFFAFCFC);
  static const Color stoneGrey = Color(0xFF91949B);
  static const Color transparent = Color(0x00000000);

  static const Color blueLight = Color(0xFFDEE7FF);
  static const Color blueMedium = Color(0xFF7FA6F9);
  static const Color blueDark = Color(0xFF1E3A78);

  static const Color yellowLight = Color(0xFFFFFFB0);
  static const Color yellowMedium = Color(0xFFFFE54D);
  static const Color yellowDark = Color(0xFFE6C200);

  static const Color greenLight = Color(0xFFE6FAF3);
  static const Color greenMedium = Color(0xFF9BE7C9);
  static const Color greenDark = Color(0xFF3C9A7A);

  static const Color pinkLight = Color(0xFFFFE0FA);
  static const Color pinkMedium = Color(0xFFFF99E0);
  static const Color pinkDark = Color(0xFFD1479C);

  static const Color greyLight = Color(0xFFE0E0E0);
  static const Color greyMedium = Color(0xFFB0B0B0);
  static const Color greyDark = Color(0xFF505050);

  static const Color sunsetOrange = Color(0xFFFF6B6B);
  static const Color lavender = Color(0xFFE6E6FA);
  static const Color teal = Color(0xFF20C997);
```

```dart
  static const Color darkText = Color(0xFFFFFFFF);
  static const Color darkTextMuted = Color(0xCCFFFFFF);

  static const Color darkPrimary = Color(0xFF2E2E35);
  static const Color darkPrimarySoft = Color(0xFF24242A);
  static const Color darkPrimaryStrong = Color(0xFF3A3A42);

  static Color primary(bool isDark) => isDark ? darkPrimary : mauveMedium;
  static Color onPrimary(bool isDark) => isDark ? darkText : Colors.white;

  static Color surface(bool isDark) => isDark ? darkSurface : Colors.white;
  static Color elevatedSurface(bool isDark) =>
      isDark ? darkElevatedSurface : Colors.white;

  static Color onSurface(bool isDark) => isDark ? darkText : mauveDark;
  static Color onSurfaceMuted(bool isDark) =>
      isDark ? darkTextMuted : mauveDark.withOpacity(.9);

  static Color scaffold(bool isDark) => isDark ? darkBg : snowWhite;

  static Color outline(bool isDark) =>
      isDark ? darkBorder : mauveDark.withOpacity(.25);

  static Color chipBg(bool isDark) =>
      isDark ? darkPrimarySoft : mauveLight.withOpacity(.35);

  static Color chipSelectedBg(bool isDark) =>
      isDark ? darkPrimaryStrong : mauveLight;

  static Color iconOnSurface(bool isDark) => isDark ? darkText : mauveDark;

  static Color danger(bool isDark) => isDark ? sunsetOrange : sunsetOrange;
}
```

Firestore_Service:

```dart
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/course.dart'; // Importing the Course model

class FirestoreService {
  FirestoreService._();
  static final instance = FirestoreService._();
  final _db = FirebaseFirestore.instance;
  final String _collection = 'courses'; // Firestore collection name

  // 1. Save or update a course (uses a Course object)
  Future<void> saveCourse({required Course course}) async {
    // Determine the document ID: use existing one (update) or generate new one (create)
    final docId = course.id ?? _db.collection(_collection).doc().id;

    // Convert the course to Map and add updatedAt timestamp
    final data = course.toJson()..['updatedAt'] = FieldValue.serverTimestamp();

    // Add createdAt only when it's a new course (course.id is null)
    if (course.id == null) {
      data['createdAt'] = FieldValue.serverTimestamp();
    }

    // Remove 'id' because it will be stored as the document ID, not as a field
    data.remove('id');

    // Save or merge the data into Firestore
    await _db.collection(_collection).doc(docId).set(
            data,
            SetOptions(merge: true),
          );
  }

  // 2. Delete a course by its ID
  Future<void> deleteCourse(String id) async {
    await _db.collection(_collection).doc(id).delete();
```

```dart
    // Remove 'id' because it will be stored as the document ID, not as a field
    data.remove('id');

    // Save or merge the data into Firestore
    await _db.collection(_collection).doc(docId).set(
            data,
            SetOptions(merge: true),
          );
  }

  // 2. Delete a course by its ID
  Future<void> deleteCourse(String id) async {
    await _db.collection(_collection).doc(id).delete();
  }

  // 3. Watch all courses and return a Stream<List<Course>>
  Stream<List<Course>> watchCourses() {
    return _db
        .collection(_collection)
        .orderBy('updatedAt', descending: true) // Sort by most recently updated
        .snapshots()
        .map(
          (snapshot) => snapshot.docs
              .map((doc) => Course.fromJson(doc.data()).copyWithId(doc.id)) // Convert each document to a Course model
              .toList(),
        );
  }
}
```